

Estructura de Datos

Act-1.3.1 Notación Asintótica (Funciones Iterativas)

Nombre: _____

Matrícula: _____

1) Contesta las preguntas en base al siguiente algoritmo

```
s = 0
for (int i=1; i<=n; i++)
    s = s + i * i
return s
```

a) ¿Qué realiza el algoritmo?

Calcula la suma de los cuadrados de los primeros n enteros

b) ¿Cuál es la operación básica?

Operación Básica: la multiplicación $i*i$ (o la suma acumulada)

c) ¿Cuántas veces se realiza la op. básica?

Se realiza n veces

d) ¿Cuál es el orden del algoritmo?

orden $O(n)$

2) ¿Cuál es el orden de cada uno de los siguientes algoritmos?

a) // Entrada: Matriz A[0..n-1, 0..n-1] de números reales.

```
for (int i=0; i<= n-2; i++)
    for (int j=i+1; j<n; j++)
        for (int k=i; k<n; k++)
            A[i,k] = A[j,k] - A[i,k] * A[j,i] / A[i,i]
```

Triple ciclo anidado (i, j, k)
cada uno recorre $\sim n$ $\rightarrow$
 $O(n^3)$

b) //Entrada: Un entero positivo (n)

```
int Q(int n){
    if (n==1) '
        return 1
    return n;
}
```

Q(n): sin ciclos ni recursión $\rightarrow O(1)$

c) //Entrada: Un entero positivo (n)

```
int P(int n){
    int acum = 0;
    if (n==0)
        return 0
    else
        if (n % 2 == 0)
            for (int i=1; i<n; i*=2)
                acum += i;
    else
        return n;
}
```

P(n): si es par, ciclo con $i*=2$ ($\approx \log_2 n$ pasos); si es impar, retorna directo $\rightarrow O(\log n)$ (peor caso)

d) //Entrada: Un entero positivo (n)

```
int a=0;
int b=n;
for (int i=1; i<= 2*n; i++) {
    a++;
    b+=a;
    c*=(a+b);
}
b=c+a;
```

Un solo ciclo $i=1..2n \rightarrow O(n)$

e) //Entrada: Un entero positivo (n)

```
int acum=1;
for (int i=1; i<=n; i++)
    for (int j=i; j<=n; j++)
        acum+=(i*j);
```

Doble ciclo $i=1..n, j=i..n$ (triangular) $\rightarrow O(n^2)$

f) //Entrada: Un entero positivo (n)

```
int b=1;
j = n;
while (j>=0) {
    b++;
    j--;
}
```

Ciclo while que decrece de n a 0 $\rightarrow O(n)$

g) //Entrada: Un entero positivo (n)

```
int acum=1;
for (int i=1; i<=n; i+=2)
    for (int j=i; j<=n; j++)
        acum+=(i*j);
```

i avanza de 2 en 2, pero el interno sigue recorriendo hasta n $\rightarrow$ sigue siendo $O(n^2)$

h) //Entrada: Un entero positivo (n)

```
int acum=1;
for (int i=1; i<=n; i*=2)
    for (int j=i; j<=n; j+=2)
        acum+=(i*j);
```

i avanza multiplicando $\times 2$ (log n pasos), interno recorre $\approx n/2$ en cada uno $\rightarrow O(n \log n)$